PROJECT DOCUMENTATION

Personal Expense Tracker

Income, Expense and Monthly Budget Management System

A web-based application for recording transactions, managing categories, monitoring monthly budgets, and visualizing personal financial activity.

Document item	Details
Project	Personal Expense Tracker
Version	1.0
Technology stack	PHP, Laravel, MySQL, SQL, HTML, CSS, JavaScript, XAMPP
Document purpose	Academic mini-project and portfolio documentation
Implementation status	Project design and implementation guide; update after testing

PREPARED BY: ROSHNI MAKWANA

AF ID: AF05329971

Document Control

This document describes the proposed design, database structure, modules, implementation steps, security considerations, and testing plan for the Personal Expense Tracker. It is written as a project guide and should be updated to match the final source code and actual test results.

Item	Details
Project name	Personal Expense Tracker
Document version	1.0
Audience	Student, evaluator, developer, interviewer
Primary purpose	Explain project requirements, design, implementation, and testing
Backend options	Laravel (recommended final implementation) or core PHP with MySQLi for practice
Database	MySQL
Local environment	XAMPP for Apache/MySQL; Laravel may run using php artisan serve
Status convention	Features described as planned until implemented and verified

How to use this document

• Use Sections 1–5 to explain the problem, objectives, scope, and architecture.

• Use Sections 6–8 to implement the database and application modules.

• Use Sections 10–12 to configure the environment and test the application.

• Replace blank student details and update the test table with observed results before submission.

Table of Contents

Section	Page (approx.)
1. Project Overview	4
2. Requirements and Scope	5
3. Technology Stack	6
4. System Architecture	7
5. Database Design	8
6. Module Description	11
7. Project Folder Structure	13
8. Development Methodology	14
9. Validation and Security	15
10. Testing Plan	16
11. Installation and Execution	18
12. Current Limitations	20
13. Future Enhancements	21
14. Conclusion	22
15. References	23

Page numbers may shift slightly depending on the PDF viewer or later edits.

1. Project Overview

1.1 Introduction

Personal Expense Tracker is a web-based financial management application that helps users record income and expenses, organize transactions into categories, set monthly spending limits, and review their financial activity through summaries and charts. The system stores records in a relational MySQL database and uses PHP/Laravel for server-side processing.

The application is intended for personal budgeting and learning purposes. It does not connect to bank accounts or provide professional financial advice. Users manually enter their transactions.

1.2 Problem Statement

When income and spending are tracked manually in notebooks or scattered spreadsheets, it can be difficult to identify spending patterns, calculate savings, or determine whether a monthly budget has been exceeded. The proposed application centralizes this information and calculates summaries automatically from saved transactions.

1.3 Objectives

• Allow users to register, log in, and maintain separate financial records.

• Create, view, edit, and delete income and expense transactions.

• Organize transactions using income and expense categories.

• Set a spending budget for each calendar month.

• Calculate total monthly income, expenses, savings, and remaining budget.

• Filter transaction records by date range and category.

• Display monthly and category-wise charts using JavaScript.

• Use database constraints and server-side validation to maintain data integrity.

1.4 Scope and Users

The primary user is an individual who wants to monitor personal finances. Each registered user should only be able to view and change records that belong to their account. A separate administrator role is not required for the initial mini-project unless added as a future enhancement.

1.5 Expected Outcome

At completion, a user should be able to log in, record transactions, manage categories, define a monthly budget, and view an updated dashboard. The figures on the dashboard and charts must be calculated from the database rather than hard-coded.

2. Requirements and Scope

2.1 Functional Requirements

ID	Requirement	Description
FR-01	Authentication	Register, log in, and log out securely.
FR-02	Transaction creation	Add income or expense with amount, date, category, and optional description.
FR-03	Transaction listing	Display the signed-in user's transactions, newest first.
FR-04	Transaction update	Edit an existing transaction owned by the signed-in user.
FR-05	Transaction deletion	Delete an owned transaction after confirmation.
FR-06	Category management	Create and view income/expense categories; rename or remove categories when safe.
FR-07	Monthly budgets	Create or update one spending budget per user per month.
FR-08	Dashboard totals	Calculate monthly income, expenses, savings, and budget remaining.
FR-09	Filters	Filter by date range and category.
FR-10	Charts	Show monthly income vs. expenses and category-wise expense distribution.
FR-11	Validation	Reject missing, invalid, negative, or incompatible transaction values.
FR-12	Data isolation	Prevent one user from accessing another user's private records.

2.2 Non-Functional Requirements

• Security: passwords are hashed and protected routes require authentication.

• Integrity: primary keys, foreign keys, unique constraints, and validation protect data consistency.

• Usability: forms use clear labels, validation messages, and responsive layouts.

• Maintainability: routes, controllers, models, views, and migrations have separate responsibilities.

• Performance: common queries filter by user and date; indexes may be added where useful.

• Portability: the project should run in a compatible PHP/Laravel environment with MySQL.

2.3 Out of Scope for the Initial Version

• Automatic bank account synchronization and payment processing.

• Investment advice, credit scoring, or tax calculations.

• Multi-currency conversion and foreign-exchange updates.

• Mobile application, cloud deployment, and third-party financial integrations.

3. Technology Stack

Technology	Role in the project
HTML5	Structures pages, forms, tables, and dashboard content.
CSS3	Styles the interface and supports responsive layouts.
JavaScript	Adds client-side interactivity, filters, and chart rendering.
PHP	Server-side language for request processing and business logic.
Laravel	Recommended PHP framework for routing, MVC structure, validation, authentication, and Eloquent ORM.
SQL	Language used to create tables, retrieve records, update data, and calculate summaries.
MySQL	Relational database that stores users, categories, transactions, and budgets.
MySQLi	PHP extension for prepared-statement database access in a separate core PHP version.
XAMPP	Local development package commonly used to run Apache and MySQL.
Composer	Installs and manages PHP/Laravel dependencies.
Chart.js (optional)	JavaScript library for pie and bar charts.
Git (recommended)	Tracks source-code changes and supports version control.

3.1 Important implementation note

Laravel normally connects to MySQL through its database layer (PDO) and uses Eloquent or the query builder. MySQLi is mainly used in a core PHP implementation. Do not mix raw MySQLi connections into Laravel controllers without a clear reason. If both approaches are part of the learning objective, build the main project in Laravel and practise the equivalent CRUD separately in core PHP.

3.2 Development environment

- Install a compatible PHP version and Composer.
- Install XAMPP and confirm the MySQL service starts successfully.
- Create the expense_tracker database using phpMyAdmin.
- Use VS Code or another code editor for development.
- Install Node.js/npm only if the chosen frontend asset workflow requires it.

4. System Architecture

4.1 Architectural Style

The Laravel implementation follows the Model–View–Controller (MVC) pattern. The browser sends a request to a route; middleware checks authentication; a controller validates input and applies business rules; a model or query builder communicates with MySQL; and a Blade view renders the response.

4.2 Request Flow

Step	Component	Responsibility
1	Browser	User submits a form or opens a page.
2	Route / middleware	Matches the URL and checks whether login is required.
3	Controller	Validates inputs, checks ownership, and coordinates the action.
4	Model / ORM	Reads or writes records using Eloquent or the query builder.
5	MySQL	Stores data and enforces relational constraints.
6	Controller / Blade	Returns a page, redirect, or validation errors.
7	Browser	Displays the updated page or message.

4.3 MVC Components

• Model: User, Category, Transaction, and Budget; defines relationships and database access.

• View: Blade templates for dashboard, transaction forms, categories, budgets, and reports.

• Controller: DashboardController, TransactionController, CategoryController, and BudgetController.

• Routes: routes/web.php maps URLs to controller methods.

• Migrations: database/migrations defines database schema changes.

4.4 Core data flow

Example: when a user adds a ■250 food expense, the request is validated, the selected category is checked, and the transaction is saved with the authenticated user's ID. The dashboard queries the current month's records and recalculates expenses and budget remaining. JavaScript receives the chart data and renders the updated visualization.

5. Database Design

The proposed database is named expense_tracker. It contains four main tables: users, categories, transactions, and budgets. The schema below is a design specification; adapt the users table if the Laravel starter kit already creates one.

5.1 Users table

Purpose: stores account details. Passwords must be hashed, not stored as plain text.

Column	Type	Constraint / purpose
id	BIGINT UNSIGNED	Primary key; auto increment
name	VARCHAR(100)	Required display name
email	VARCHAR(150)	Required and unique
password	VARCHAR(255)	Required password hash
created_at	TIMESTAMP NULL	Account creation time
updated_at	TIMESTAMP NULL	Last update time

5.2 Categories table

Column	Type	Constraint / purpose
id	BIGINT UNSIGNED	Primary key; auto increment
user_id	BIGINT UNSIGNED NULL	Owner; NULL can represent shared default category
name	VARCHAR(100)	Required category name
type	ENUM('income','expense')	Defines compatible transaction type
created_at / updated_at	TIMESTAMP NULL	Record timestamps

5.3 Transactions table

Column	Type	Constraint / purpose
id	BIGINT UNSIGNED	Primary key; auto increment
user_id	BIGINT UNSIGNED	Foreign key to users.id
category_id	BIGINT UNSIGNED	Foreign key to categories.id
type	ENUM('income','expense')	Income or expense
amount	DECIMAL(12,2)	Positive monetary value
description	VARCHAR(255) NULL	Optional note
transaction_date	DATE	Date of transaction
created_at / updated_at	TIMESTAMP NULL	Record timestamps

5.4 Budgets table

Column	Type	Constraint / purpose
id	BIGINT UNSIGNED	Primary key; auto increment
user_id	BIGINT UNSIGNED	Foreign key to users.id
month	DATE	Store first day of target month
amount	DECIMAL(12,2)	Positive spending limit
created_at / updated_at	TIMESTAMP NULL	Record timestamps
Unique key	(user_id, month)	One budget per user per month

5.5 Sample records

Illustrative sample records for one user in October 2026:

Transaction	Type	Category	Amount	Date
Monthly salary	Income	Salary	■20,000.00	2026-10-01
Lunch	Expense	Food	■250.00	2026-10-02
Local travel	Expense	Transport	■100.00	2026-10-03
Groceries	Expense	Food	■1,200.00	2026-10-04

5.6 Relationships

• One user has many transactions; each transaction belongs to one user.

• One user has many personal categories; shared default categories may have a NULL user_id.

• One category can be referenced by many transactions.

• One user can have many budgets, but the unique user/month constraint allows only one per month.

5.7 Constraints and integrity

• PRIMARY KEY uniquely identifies each row.

• FOREIGN KEY connects related tables and prevents orphaned references.

• UNIQUE prevents duplicate email addresses and duplicate monthly budgets for the same user.

• NOT NULL requires a value; CHECK (amount > 0) rejects non-positive amounts on MySQL versions that enforce CHECK constraints.

• Use RESTRICT for deleting categories that are still referenced by transactions; provide a safe archive or reassignment workflow.

5.8 Relationship diagram (text form)

```
users (1) --------< transactions >-------- (1) categories
  |                       |
  |                       | category_id
  | user_id               | user_id
  |                       |
  +-----------< budgets   +-- transaction_date, type, amount

users (1) --------< categories (personal categories)
users (1) --------< budgets (one per month)
```

5.9 Database setup SQL

```
CREATE DATABASE IF NOT EXISTS expense_tracker;
USE expense_tracker;

CREATE TABLE users (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  created_at TIMESTAMP NULL DEFAULT NULL,
  updated_at TIMESTAMP NULL DEFAULT NULL
) ENGINE=InnoDB;

CREATE TABLE categories (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT UNSIGNED NULL,
  name VARCHAR(100) NOT NULL,
  type ENUM('income','expense') NOT NULL,
  created_at TIMESTAMP NULL DEFAULT NULL,
  updated_at TIMESTAMP NULL DEFAULT NULL,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE transactions (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT UNSIGNED NOT NULL,
  category_id BIGINT UNSIGNED NOT NULL,
  type ENUM('income','expense') NOT NULL,
  amount DECIMAL(12,2) NOT NULL,
  description VARCHAR(255) NULL,
  transaction_date DATE NOT NULL,
  created_at TIMESTAMP NULL DEFAULT NULL,
  updated_at TIMESTAMP NULL DEFAULT NULL,
  CONSTRAINT chk_transaction_amount CHECK (amount > 0),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE RESTRICT,
  INDEX idx_user_date (user_id, transaction_date),
  INDEX idx_user_category (user_id, category_id)
) ENGINE=InnoDB;

CREATE TABLE budgets (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT UNSIGNED NOT NULL,
  month DATE NOT NULL,
  amount DECIMAL(12,2) NOT NULL,
  created_at TIMESTAMP NULL DEFAULT NULL,
  updated_at TIMESTAMP NULL DEFAULT NULL,
  CONSTRAINT chk_budget_amount CHECK (amount > 0),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  UNIQUE KEY uq_user_month (user_id, month)
) ENGINE=InnoDB;
```

Implementation note: Laravel projects should create these tables through migrations. If a default users migration exists, modify it instead of executing a second users-table definition. The category ownership and transaction-category compatibility rules must also be checked in application logic.

6. Module Description

6.1 Authentication

Provides registration, login, and logout. Authenticated routes protect private transaction and budget pages. Laravel authentication tools should be used where available. Passwords are stored using secure hashing.

6.2 Dashboard

Displays the selected month's income, expenses, savings, budget amount, amount spent, remaining budget, recent transactions, and charts. The dashboard must filter every financial query by the authenticated user's ID.

6.3 Transaction Management

• Create: validate transaction type, positive amount, category, date, and optional description.

• Read: list the user's transactions in descending date order.

• Update: verify ownership and revalidate the modified fields.

• Delete: verify ownership and request confirmation before deletion.

• Filter: support date range and category filters.

6.4 Category Management

Users can create and manage categories such as Food, Transport, Rent, Shopping, Salary, Freelancing, and Business. Each category has a type. A transaction's type must match its category's type. Shared default categories can be offered to all users, while personal categories are owned by one user.

6.5 Monthly Budget Management

Users set a monthly spending limit. The application calculates expenses for the selected month and compares them with the budget. Budget remaining = budget amount – eligible expenses. A negative result indicates the budget has been exceeded. Budget progress percentage = expenses ÷ budget amount × 100.

6.6 Reports and Charts

• Bar chart: compare income and expenses by month.

• Pie/doughnut chart: show expense distribution by category.

• Date-range report: show transactions for a selected period.

• Summary report: show income, expenses, savings, budget and budget usage.

6.7 Example dashboard calculations

Metric	Example value	Calculation
Total income	■20,000	Sum of income transactions in selected month
Total expenses	■5,500	Sum of expense transactions in selected month
Savings	■14,500	■20,000 – ■5,500
Monthly budget	■8,000	Saved monthly budget
Budget remaining	■2,500	■8,000 – ■5,500
Budget used	68.75%	■5,500 ÷ ■8,000 × 100

Savings and remaining budget are different measures. Savings compares income with expenses; budget remaining compares the spending limit with expenses.

7. Project Folder Structure

7.1 Suggested Laravel structure

```
expense-tracker/
■■■ app/
■   ■■■ Models/
■   ■   ■■■ User.php
■   ■   ■■■ Category.php
■   ■   ■■■ Transaction.php
■   ■   ■■■ Budget.php
■   ■■■ Http/Controllers/
■       ■■■ DashboardController.php
■       ■■■ TransactionController.php
■       ■■■ CategoryController.php
■       ■■■ BudgetController.php
■■■ database/
■   ■■■ migrations/
■   ■■■ seeders/
■■■ resources/
■   ■■■ views/
■   ■   ■■■ layouts/
■   ■   ■■■ dashboard.blade.php
■   ■   ■■■ transactions/
■   ■   ■■■ categories/
■   ■   ■■■ budgets/
■   ■   ■■■ reports/
■   ■■■ css/
■   ■■■ js/
■■■ routes/web.php
■■■ public/
■■■ .env
■■■ composer.json
```

7.2 Folder responsibilities

Path	Responsibility
app/Models	Eloquent models, relationships, and database-facing logic.
app/Http/Controllers	Request processing, validation, ownership checks, and workflows.
database/migrations	Versioned database schema changes.
database/seeders	Optional default categories and demo data.
resources/views	Blade templates for pages and forms.
resources/css and resources/js	Styles and frontend interactivity; may be bundled by Vite.
routes/web.php	URL-to-controller mapping and authentication middleware.
public	Public entry point and publicly served assets.
.env	Local database settings and secrets; never commit secrets.

7.3 Core PHP alternative

If the assignment requires MySQLi explicitly, a separate core PHP version may use folders such as config/db.php, includes/header.php, includes/footer.php, transactions/, categories/, budgets/, assets/css/, and assets/js/. Use MySQLi prepared statements and session checks. This is an alternative structure, not the standard Laravel structure.

8. Development Methodology

The project can be developed incrementally. Each module should be implemented, tested, and committed before moving to the next module. This makes errors easier to isolate and gives a clear record of progress.

8.1 Suggested development phases

Phase	Work items	Deliverable
1. Planning	Requirements, page list, data model	Project specification
2. Setup	Install PHP, Composer, XAMPP, Laravel	Working local environment
3. Database	Create migrations and relationships	Four related tables
4. Authentication	Registration, login, logout, protected routes	Private user area
5. Core CRUD	Transactions and categories	Working forms and lists
6. Budgeting	Monthly budget create/update and calculations	Budget status page
7. Dashboard	Totals, filters, charts	Financial summary
8. Validation	Security and manual test cases	Test evidence
9. Documentation	Screenshots, limitations, conclusion	Submission-ready report

8.2 Version control

• Initialize Git and commit the project before major changes.

• Use meaningful commit messages such as 'Add transaction CRUD' or 'Add monthly budget validation'.

• Keep .env, passwords, and application secrets out of the repository.

• Do not commit generated dependencies or personal financial data.

8.3 Data and migration consistency

Use migrations as the authoritative schema history in Laravel. Test migrations on a fresh local database before submission. If the schema changes, create a new migration rather than manually changing only the local database. Back up any data before running destructive migration commands.

9. Validation and Security

9.1 Authentication and sessions

• Require login for dashboard, transactions, categories, budgets, and reports.

• Hash passwords using Laravel's supported password hashing tools.

• Regenerate sessions after successful login and invalidate the session on logout.

• Consider login rate limiting and email verification if appropriate for the final scope.

9.2 Authorization and ownership

Every read, edit, and delete action must be scoped to the authenticated user's records. Never trust a user_id supplied by a form. Obtain the user ID from the authenticated session and verify that a selected category is either a shared default category or belongs to that user.

9.3 Input validation

• Transaction type must be income or expense.

• Amount must be numeric and greater than zero; use DECIMAL in MySQL for monetary values.

• Category must exist, be accessible to the user, and match the transaction type.

• Transaction date must be a valid date.

• Description should have a reasonable maximum length.

• Budget amount must be positive and the month should be normalized to the first day of that month.

9.4 Database and web protections

• Use Laravel Eloquent/query builder or parameterized SQL; do not concatenate raw user input into SQL.

• Use CSRF protection on state-changing forms.

• Escape displayed user-generated text to reduce cross-site scripting risks.

• Apply foreign keys and unique constraints in the database.

• Avoid exposing .env contents, database credentials, stack traces, or debug mode in production.

• Use HTTPS and secure environment configuration for any deployed version.

9.5 Financial calculation rules

Calculate totals from the transaction table for the selected user and period. Do not let client-side JavaScript decide authoritative financial totals. JavaScript should present values returned by the backend; the backend and database remain the source of truth.

10. Testing Plan

The following cases should be executed manually or automated where possible. The result/status column is intentionally left as a template: mark each case Pass or Fail only after observing the application.

ID	Test scenario	Expected result	Actual/status
TC-01	Register with valid details	Account is created and can log in	To be recorded
TC-02	Register with duplicate email	Validation rejects duplicate email	To be recorded
TC-03	Login with incorrect password	Access is denied with a safe error	To be recorded
TC-04	Open dashboard while logged out	Redirect to login or deny access	To be recorded
TC-05	Add valid income	Record saves; income total increases	To be recorded
TC-06	Add valid expense	Record saves; expense total increases	To be recorded
TC-07	Submit zero or negative amount	Validation rejects the value	To be recorded
TC-08	Edit an owned transaction	Changes persist and totals update	To be recorded
TC-09	Delete an owned transaction	Record is removed and totals recalculate	To be recorded
TC-10	Attempt to edit another user's record	Request is denied	To be recorded
TC-11	Use category of wrong type	Validation rejects the category	To be recorded
TC-12	Set monthly budget	Budget saves for selected month	To be recorded
TC-13	Set same month budget again	Existing budget updates, no duplicate	To be recorded
TC-14	Filter by date range	Only matching transactions appear	To be recorded
TC-15	Filter by category	Only matching category records appear	To be recorded
TC-16	Check savings calculation	Income minus expenses is correct	To be recorded
TC-17	Check budget remaining	Budget minus expenses is correct	To be recorded
TC-18	Check chart data	Chart matches backend totals	To be recorded
TC-19	Submit invalid form/CSRF request	Request is safely rejected	To be recorded
TC-20	Delete category used by a transaction	Deletion is blocked or safely handled	To be recorded

10.1 Test evidence

For each test, record the date, test data, steps, observed result, and pass/fail status. Screenshots may be included in the final report, but they should not expose passwords, real financial details, or other private information.

11. Installation and Execution

11.1 Prerequisites

• A compatible PHP version for the selected Laravel release.

• Composer installed and available in the terminal.

• XAMPP installed with MySQL available locally.

• VS Code or another code editor.

• Node.js/npm if frontend assets need to be compiled.

11.2 Setup steps for Laravel

1. Start MySQL in XAMPP. Apache is needed if you plan to serve the app through XAMPP; Laravel's built-in development server can run separately.

2. Open phpMyAdmin at http://localhost/phpmyadmin and create a database named expense_tracker.

3. Open a terminal and create the project:

```
composer create-project laravel/laravel expense-tracker\ncd expense-tracker
```

4. Configure the .env database settings. Typical local settings are shown below; change them if your MySQL username, password, or port differs.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=expense_tracker
DB_USERNAME=root
DB_PASSWORD=
```

5. Create the models, migrations, and controllers:

```
php artisan make:model Category -m
php artisan make:model Transaction -m
php artisan make:model Budget -m
php artisan make:controller DashboardController
php artisan make:controller TransactionController --resource
php artisan make:controller CategoryController --resource
php artisan make:controller BudgetController --resource
```

6. Define the table schemas and relationships in the migrations, then run:

```
php artisan migrate
```

7. Implement authentication, routes, controllers, Blade views, validation, and charts. If the project already has a default users migration, modify it rather than creating a duplicate users table.

8. Start the application:

```
php artisan serve
```

9. Open the local URL printed by Laravel, commonly http://127.0.0.1:8000, and test the main workflows.

11.3 Useful commands

Command	Purpose
php artisan serve	Start the local Laravel development server.
php artisan migrate	Run pending migrations.
php artisan migrate:status	View migration status.
php artisan route:list	List registered routes.
php artisan optimize:clear	Clear cached framework configuration and routes.
php artisan make:model Transaction -m	Create a model and migration.
composer install	Install PHP dependencies listed in composer files.
npm install	Install frontend dependencies when package.json is present.
npm run build	Build frontend assets when configured.

11.4 XAMPP and Laravel note

XAMPP's MySQL service supplies the database. Laravel's php artisan serve command supplies a separate development web server. Avoid trying to use the same port for both Apache and the Laravel server. If a database connection fails, verify that MySQL is running, the port is correct, and the .env settings match the local database.

11.5 Environment safety

Never share the .env file, application key, database password, or real user credentials. Keep debug mode for local development only and do not upload private financial records to a public repository.

12. Current Limitations

The following limitations apply to the initial proposed scope. Update this section after implementation to reflect the actual state of the application.

• Transactions are entered manually; there is no bank synchronization.

• The first version focuses on one currency (Indian Rupees) and does not perform currency conversion.

• The application is intended for local development and academic demonstration until deployment and security review are completed.

• Charts depend on correct backend aggregation and should be checked against SQL totals.

• PDF/CSV exports, recurring transactions, savings goals, and notifications are not part of the core version.

• A separate core PHP/MySQLi implementation and Laravel implementation should not be treated as one shared database access layer.

12.1 Data-model considerations

• A transaction category must belong to the same user or be a shared default category.

• A category's type must match the transaction type.

• The monthly budget should be unique for each user and month.

• Use a safe policy for category deletion when transactions reference that category.

• Avoid hard-deleting users or financial history unintentionally; consider archival or account deletion rules if the project grows.

13. Future Enhancements

• Export monthly summaries and filtered transactions to PDF or CSV.

• Add recurring expenses such as rent, subscriptions, and utility bills.

• Create savings goals with target amounts and progress indicators.

• Send alerts when spending reaches a configurable percentage of the budget.

• Add more advanced date comparisons and month-to-month analytics.

• Add a dark mode and improve mobile accessibility.

• Add pagination and server-side search for large transaction histories.

• Add automated feature tests for authorization, validation, and financial calculations.

• Add optional multi-currency support with clearly defined exchange-rate sources.

• Deploy to a secure hosting environment with backups, HTTPS, monitoring, and a documented recovery plan.

13.1 Priority order

Priority	Enhancement	Reason
High	Authorization and automated tests	Protect user records and catch regressions.
High	PDF/CSV reports	Useful for practical demonstrations and record keeping.
Medium	Recurring transactions	Reduces repetitive manual entry.
Medium	Budget alerts	Helps users notice high spending earlier.
Low / optional	Multi-currency and integrations	Adds complexity and external dependencies.

14. Conclusion

The Personal Expense Tracker is a practical web application designed to help users record income and expenses, manage categories, set monthly spending limits, and review their financial activity. Its core design uses four relational tables—users, categories, transactions, and budgets—and demonstrates CRUD operations, SQL aggregation, foreign-key relationships, validation, and dashboard reporting.

Using Laravel for the main implementation provides a clear MVC structure, routing, authentication integration, Eloquent relationships, and server-side validation. MySQL stores the financial records, while HTML and CSS create the interface and JavaScript presents interactive charts. XAMPP can provide the local MySQL service during development.

The project should be considered complete only after the main workflows have been implemented, database totals have been verified, security and ownership checks have been tested, and actual test results have been recorded. Future improvements can include exports, recurring transactions, savings goals, notifications, and secure deployment.

Suggested demonstration sequence

• Log in to a test account.

• Add one income record and two expense records.

• Show the dashboard totals and savings calculation.

• Set a monthly budget and demonstrate remaining budget.

• Filter transactions by date and category.

• Edit and delete a test transaction and show that totals update.

• Show the database tables and explain their relationships.

15. References

• Laravel Documentation — https://laravel.com/docs

• Laravel Eloquent ORM — https://laravel.com/docs/eloquent

• Laravel Validation — https://laravel.com/docs/validation

• Laravel Authentication — https://laravel.com/docs/authentication

• MySQL Reference Manual — https://dev.mysql.com/doc/

• PHP Documentation — https://www.php.net/docs.php

• PHP MySQLi Documentation — https://www.php.net/manual/en/book.mysqli.php

• Composer Documentation — https://getcomposer.org/doc/

• XAMPP Documentation / FAQs — https://www.apachefriends.org/

• Chart.js Documentation — https://www.chartjs.org/docs/

Appendix A — Key formulas

Metric	Formula
Total income	Sum of amounts where type = income for the selected user and period.
Total expenses	Sum of amounts where type = expense for the selected user and period.
Savings	Total income – total expenses.
Budget remaining	Monthly budget amount – eligible monthly expenses.
Budget usage (%)	(Eligible monthly expenses ÷ monthly budget amount) × 100, when budget > 0.

Appendix B — Example SQL aggregation

```
SELECT
  COALESCE(SUM(CASE WHEN type = 'income'
                    THEN amount ELSE 0 END), 0) AS total_income,
  COALESCE(SUM(CASE WHEN type = 'expense'
                    THEN amount ELSE 0 END), 0) AS total_expenses
FROM transactions
WHERE user_id = ?
  AND transaction_date >= ?
  AND transaction_date < ?;
```

Use parameter binding for the user ID and date range. For a month, the end date should be the first day of the following month so that all dates in the selected month are included without relying on a hard-coded last day.

End of document